

Paralelní algoritmus pro řešení problému pokrytí mřížky quatrominy

Matěj Severýn

FIT ČVUT, Thákurova 9, 160 00 Praha 6

29. dubna 2026

1 Definice problému a popis sekvenčního algoritmu

Úlohou je nalézt pokrytí obdélníkové herní desky S o rozměrech $a \times b$ s minimální cenou. Pro rozměry desky platí $3 \leq a, b \leq 20$ a celková plocha $ab \geq 15$. Každé políčko desky je ohodnoceno přirozeným číslem z intervalu $\langle 1, 100 \rangle$.

K pokrytí se využívají výhradně quatromina typu „T“ a „Z“, která lze při vkládání libovolně otáčet a překlápět. Pokrytí je definováno jako nepřekrývající se umístění těchto quatromin takové, že do zbylého prostoru již žádné další quatromino vložit nelze. Některá políčka tak mohou zůstat nepokrytá. Současně platí striktní omezení na paritu: počet použitých dílků typu T a typu Z se musí rovnat, nebo se smí lišit maximálně o 1.

Cena pokrytí je definována jako součet ohodnocení všech nepokrytých políček na desce. Cílem algoritmu je minimalizovat tuto cenu.

Výstupem algoritmu je maticový popis výsledného řešení, kde každé umístěné quatromino je popsáno textovým identifikátorem (např. T1, Z2) s unikátním pořadovým číslem. Nepokrytá políčka jsou vypsaná svou původní číselnou hodnotou.

1.1 Sekvenční algoritmus

Sekvenční algoritmus řeší problém prohledáváním stavového prostoru metodou větví a mezí. Jedná se o algoritmus typu **BB-DFS** (Branch & Bound - Depth First Search).

- **Stavový prostor a větvení:** Na počátku jsou všechna políčka nerozhodnutá. Deska se zaplňuje systematicky (zleva doprava, shora dolů). Algoritmus najde nejbližší nerozhodnuté políčko a zkusí ho buď pokrýt jedním z platných natočení quatromina, nebo jej označí jako trvale neobsazené. Algoritmus upřednostňuje kroky, které minimalizují výslednou cenu.
- **Ořezávání (Pruning):** K prořezávání neperspektivních větví dochází ve dvou případech:
 1. Pokud je aktuální cena (součet hodnot již trvale neobsazených políček v aktuální větvi) větší nebo rovna dosavadnímu nalezenému minimu (*best_cost*).
 2. Pokud na základě zbývajících počtu volných políček již není fyzicky možné splnit podmínku parity (počet použitých T a Z dílků se bude lišit o více než 1).
- **Triviální dolní mez a ukončení výpočtu:** Triviální dolní mez je spočítána jako součet k nejmenších hodnot na desce, kde $k = (a \times b) \pmod{4}$. Pokud je ab dělitelné čtyřmi, je triviální mez 0. Netriviální dolní mez algoritmus nepoužívá. Jakmile algoritmus najde pokrytí, jehož cena se rovná triviální dolní mezi, je výpočet okamžitě ukončen jako optimální.

2 Popis paralelního algoritmu OpenMP – taskový paralelismus

Tasková implementace dynamicky rozděluje práci pomocí asynchronních úkolů přes direktivu `#pragma omp task`. Algoritmus začíná v jednom vlákně (`#pragma omp single`), které vygeneruje prvotní úkol nad prázdnou deskou.

Hlavní myšlenka spočívá v paralelním zpracování větví stavového stromu. Ve funkci `solve_dfs_task` se při každém úspěšném pokusu o umístění dílku (T , Z , nebo vynechání políčka) vytvoří nový task. Aby se zabránilo *race conditions* (souběhům nad sdílenými daty), musí být do každého tasku předána plná hluboká kopie aktuálního stavu desky (`vector<vector<int>> board`) a historie použitých dílků (`vector<char> p_types`). Jediné globálně sdílené proměnné jsou nejlepší nalezená cena (`best_cost`) a flag indikující dosažení triviální meze (`optimal_found`), k nimž se přistupuje výhradně přes synchronizační direktivy `#pragma omp atomic` a `#pragma omp critical`.

Aby nedocházelo k exponenciálnímu zahlcení paměti obrovským množstvím drobných úkolů, je větvení omezeno prahovým parametrem `TASK_CUTOFF`. Pokud aktuální hloubka prohledávání (index políčka `cell_idx`) dosáhne této hodnoty, vlákno již negeneruje nové tasky, ale přejde do klasické čistě sekvenční rekurze (`solve_dfs_seq`), která předává pole odkazem a šetří paměť. Tímto přístupem se vytváří hrubší zrnitost výpočtu (*coarse granularity*). Na konci paralelní tvorby tasků nad daným políčkem čeká `#pragma omp taskwait` na jejich dokončení.

3 Popis paralelního algoritmu OpenMP – datový paralelismus

Datový paralelismus přistupuje k problému odlišně – odstraňuje obrovskou režii z kopírování vektorů. Rozděluje výpočet do dvou striktně oddělených fází: prohledávání do šířky (BFS) a paralelního prohledávání do hloubky (DFS).

1. **Generování stavů (Sekvenční BFS):** Na začátku programu projde sekvenčně hlavní vlákno stavový prostor pomocí fronty (`std::queue`). Přikládá dílky na volná políčka a generuje částečně rozpracované validní stavy desky. Toto běží, dokud není ve frontě vygenerován požadovaný počet uzlů definovaný parametrem `EnoughStates`. Vygenerované stavy jsou následně přesunuty z fronty do statického vektoru (`initial_states`).
2. **Paralelní výpočet (DFS):** Následně se otevře paralelní blok `#pragma omp parallel for schedule(dynamic)`, který prochází vektor počátečních stavů. Plánovač `dynamic` zajistí, že si volná vlákna asynchronně odebírají stavy k dořešení. Každé vlákno si vezme unikátní částečně rozpracovaný stav a pustí na něj čistě sekvenční rekurzi (`solve_dfs_seq`). Vzhledem k tomu, že má každé vlákno svůj vlastní, hluboce prohledávaný podstrom, nevyžaduje samotná DFS rekurze žádnou hlubokou kopii dat a deska se předává pouze odkazem. Zajištění zápisu nalezeného globálního minima funguje stejně jako u tasků (přes `atomic read` a `critical` blok).

Tento model garantuje excelentní paměťovou efektivitu, protože vlákna alokují paměť pouze jednou na začátku (přijetím stavu z vektoru) a veškerou další práci řeší *in-place* odkazem na svůj lokální zásobník.

4 Popis paralelního algoritmu v MPI

MPI implementace využívá hierarchickou architekturu **Master-Slave** kombinující zasílání zpráv (MPI) a sdílenou paměť (OpenMP na každém uzlu). Algoritmicky se jedná o dvoustupňový distribuovaný datový paralelismus.

Základní princip komunikace a rozdělení práce je následující:

- **Master (uzel 0):** Pomocí sekvenčního prohledávání do šířky (BFS) do definované hloubky vygeneruje prvotní frontu rozpracovaných stavů. Následně funguje jako dispečer: odesílá tyto stavy pracovním uzlům (*Slaves*) a sbírá od nich výsledky. Udržuje globální minimum (*best_cost*). Jakmile si volný Slave uzel řekne o další práci, Master k novému stavu rovnou přibalí i aktuální hodnotu globálního minima, čímž se ořezávací mez šíří napříč klastrem.
- **Slave (ostatní uzly):** Uzel přijme od Mastera jeden rozpracovaný stav a s ním i informaci o nejlepším dosud nalezeném řešení v klastru. Přijatý stav nerozpracovává sekvenčně, ale lokálně jej pomocí dalšího BFS rozštěpí na sadu desítek až stovek menších pod-stavů. Tyto stavy jsou následně rozděleny mezi lokální OpenMP vlákna uzlu pomocí direktivy `#pragma omp parallel for schedule(dynamic)`. Každé vlákno vezme pod-stav a provede nad ním čistě sekvenční DFS výpočet s lokálním ořezáváním. Po dokončení všech lokálních vláken odevzdá Slave uzel svůj nejlepší nalezený výsledek Masterovi a čeká na další zprávu s úkolem.

Kritické úpravy pro efektivitu a škálovatelnost: Zásadním prvkem navrženého řešení je dvoustupňová granularita generování stavů (Master BFS a Slave BFS). Tento přístup zajišťuje, že se po síti neodesílá zbytečně mnoho drobných zpráv (předejití zahlcení sítě a Mastera), ale zároveň Slave uzly přijímají dostatečně obsáhlé uzly stavového prostoru, aby si v nich samy vygenerovaly práci pro svých plných 48 vláken. Pro správné škálování v prostředí klastru bylo také nezbytné při spouštění úlohy vypnout zamykání procesů na jádra nastavením systémové proměnné `MV2_ENABLE_AFFINITY=0`, aby nedocházelo ke kolizi MPI procesu a OpenMP vláken.

5 Naměřené výsledky a experimenty

Pro komplexní otestování a vyhodnocení paralelizace algoritmu jsme využili dvě sady instancí. První sadu tvoří 11 základních testovacích map ze zadání. Tyto mapy se vyznačují malou velikostí stavového prostoru a jejich sekvenční doba běhu se pohybuje v řádech od zlomků milisekund po desítky sekund. Druhou sadu tvoří 3 vlastní generované, mnohem složitější mapy s příponou `_hard.txt`, u kterých dosahuje sekvenční čas prohledávání i několika minut. Čas potřebný pro načítání vstupu z disku a výpisy na standardní výstup nebyl do měření zahrnut.

5.1 Metodika měření a ladění parametrů

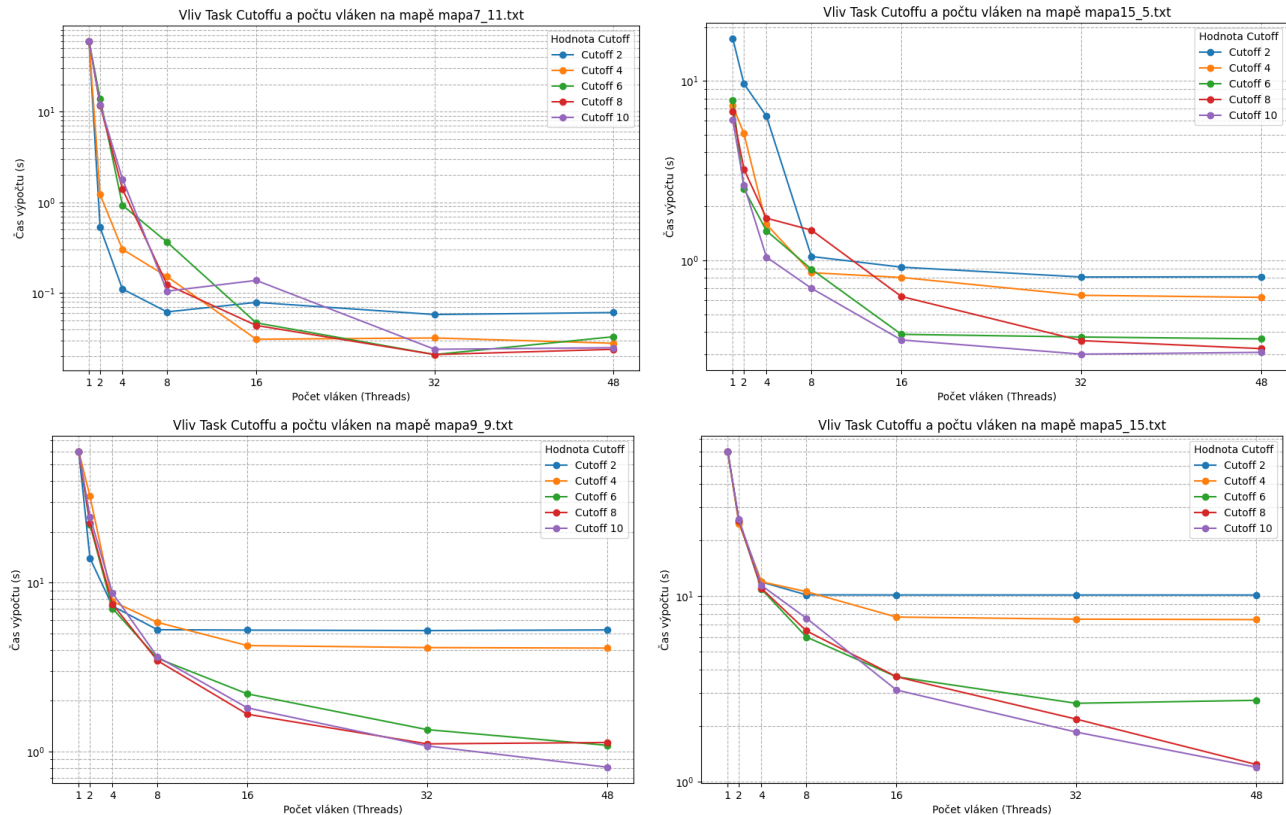
Před zahájením hlavního měření bylo nutné pro každou architekturu stanovit optimální hodnoty ladicích parametrů. Pro tento předvýběr byly zvoleny 4 nejtěžší instance ze základní sady zadání (`mapa7_11.txt`, `mapa15_5.txt`, `mapa9_9.txt` a `mapa5_15.txt`) a provedena parametrická analýza. Na základě této analýzy a dalších experimentů byly pro finální běhy stanoveny konkrétní hodnoty.

5.1.1 Taskový paralelismus: `TASK_CUTOFF`

Parametr `TASK_CUTOFF` určuje maximální hloubku zanoření ve stromu, do které se asynchronně vytvářejí nové OpenMP tasky. Příliš nízký cutoff vede k malému množství velkých úkolů (špatný

load balancing), zatímco příliš vysoký cutoff způsobuje masivní zahlcení fronty tasků statisíci drobných úkolů, což generuje enormní paměťovou a plánovací režii.

Zvolené nastavení: Na základě měření (viz Obrázek 1) byla pro jednoduché úlohy stanovena hodnota **8**, zatímco u časově náročných (těžkých) úloh bylo nutné omezit tvorbu tasků a cutoff byl snížen na hodnotu **2**.



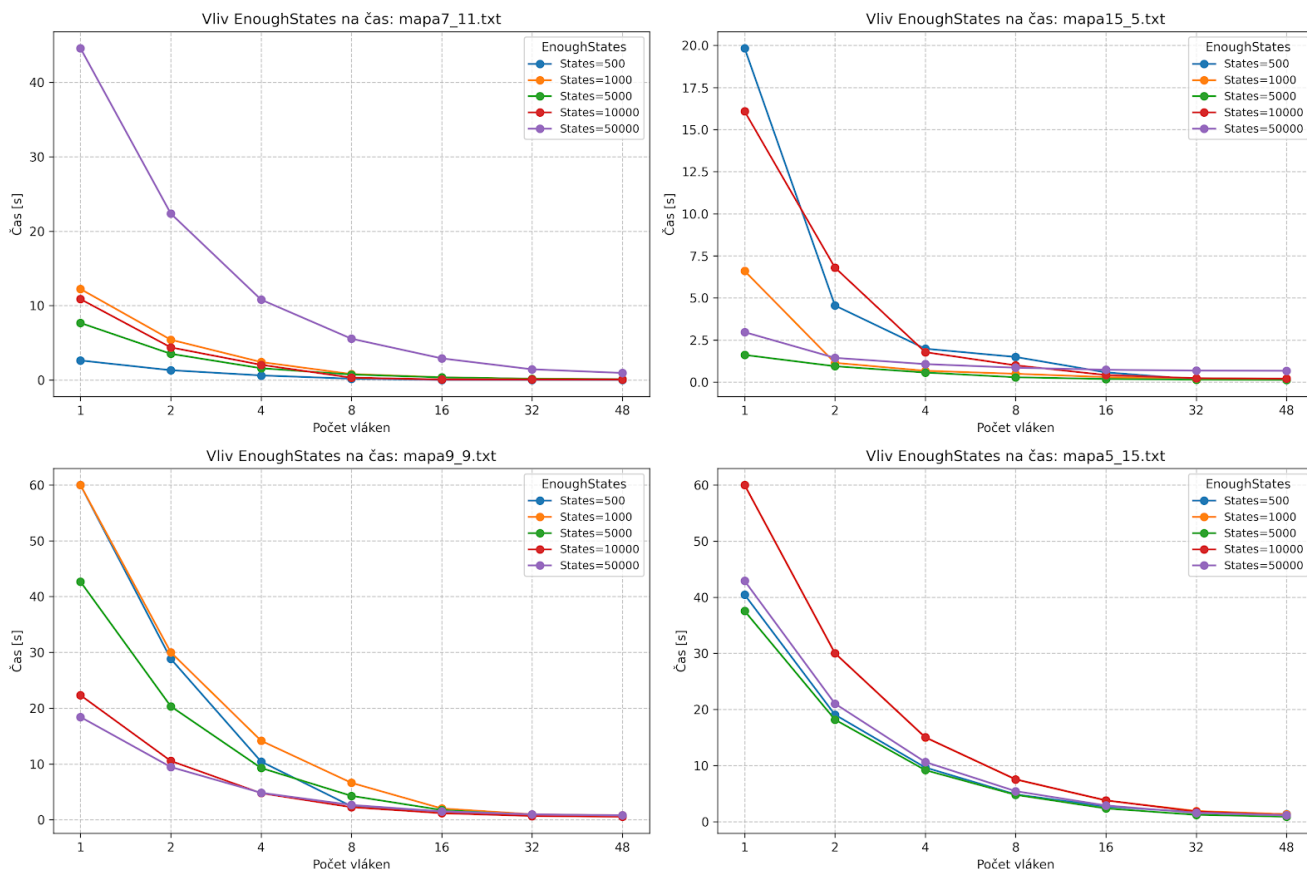
Obrázek 1: Analýza vlivu parametru TASK_CUTOFF na časovou složitost u 4 vybraných instancí.

5.1.2 Datový paralelismus: EnoughStates

Parametr **EnoughStates** definuje limit počtu rozpracovaných stavů (vrcholů), které sekvenční Master vlákno vygeneruje pomocí prohledávání do šířky (BFS) před spuštěním paralelní **for** sekce. Nízký počet způsobuje předčasné vyhladovění vláken, extrémně vysoký počet naopak zbytečnou počáteční alokaci.

Zvolené nastavení: Z analýzy (viz Obrázek 2) a charakteristik stavového stromu byly hodnoty nastaveny adaptivně podle obtížnosti instance:

- Jednoduché úlohy a `mapa7_12_hard.txt`: **5 000** stavů.
- Pro `mapa9_10_hard.txt`: **20 000** stavů.
- Nejtěžší `mapa10_10_hard.txt`: **50 000** stavů.



Obrázek 2: Analýza vlivu limitu fronty BFS (EnoughStates) na celkový čas běhu u datového paralelismu.

5.1.3 MPI architektura: Granularita Master/Slave

V distribuovaném prostředí bylo nutné optimalizovat velikost balíků práce posílaných po síti a následně štěpených na uzlech.

Zvolené nastavení: Pro jednoduché úlohy Master generoval **500** stavů a každý Slave lokálně **100** pod-stavů. Pro složité úlohy (`_hard.txt`) byla granularita zjemněna, Master generoval **5 000** stavů pro efektivní distribuci napříč sítí a každý Slave z přijatého stavu vygeneroval **150** dalších stavů pro svých 48 lokálních OpenMP vláken. Počet úvodních stavů pro datový paralelismus běžící na samotném uzlu (dataMPI) byl shodný s hodnotami OpenMP DATA (tj. 5k, 20k, 50k).

5.2 Sekvenční algoritmus

Před vyhodnocením paralelizace byly všechny instance změřeny pomocí základního sekvenčního algoritmu. Tyto časy tvoří referenční *baseline* pro výpočet zrychlení.

Tabulka 1: Naměřené časy sekvenčního algoritmu pro všechny testované instance

Instance (mapa)	Čas [s]
Základní instance	
mapa3_5.txt	0.0001
mapa5_5.txt	0.0016
mapa7_7.txt	0.0048
mapa3_11.txt	0.0189
mapa4_15.txt	0.0854
mapa7_10.txt	1.0406
mapa5_11.txt	1.0441
mapa7_11.txt	8.3129
mapa15_5.txt	24.6503
mapa9_9.txt	28.6691
mapa5_15.txt	44.1724
Náročné instance	
mapa9_10_hard.txt	120.0290
mapa7_12_hard.txt	200.7910
mapa10_10_hard.txt	328.8200

Analýza: Tabulka 1 odhaluje strmý růst složitosti, který nezávisí jen na ploše desky, ale silně i na jejím poměru stran a topologii. Důkazem jsou instance `mapa15_5.txt` (24.65 s) a `mapa5_15.txt` (44.17 s) se shodnou rozlohou, avšak odlišným časem. Tato divergence plyne z fixního směru prohledávání (shora dolů, zleva doprava), tvar mřížky určuje šířku úvodních vrstev stavového stromu a rychlost nalezení prvních ořezávacích mezí. Extrémní nárůst časů od zlomků sekund po minuty u těžkých čtvercových map tak poskytuje ideálně heterogenní testovací sadu.

5.3 Základní testy napříč všemi implementacemi

Ačkoliv většina základních instancí trvala sekvenčně jen velmi krátce, byly změřeny ve všech třech implementacích. V následujících tabulkách prezentujeme kompletní výsledky pro všechny mapy z této základní sady, seřazené podle sekvenční náročnosti.

Tabulka 2: Základní instance: OpenMP Datový paralelismus [s]

Instance	2 vl.	4 vl.	8 vl.	12 vl.	16 vl.	32 vl.	48 vl.
mapa3_5.txt	0.0105	0.0108	0.0114	0.0121	0.0124	0.0140	0.0159
mapa5_5.txt	0.0582	0.0572	0.0544	0.0545	0.0546	0.0554	0.0566
mapa7_7.txt	0.1290	0.0997	0.0756	0.0690	0.0663	0.0635	0.0635
mapa3_11.txt	0.0461	0.0419	0.0373	0.0375	0.0366	0.0372	0.0626
mapa4_15.txt	0.0493	0.0472	0.0405	0.0404	0.0406	0.0414	0.0808
mapa7_10.txt	0.6063	0.2837	0.1607	0.1270	0.1110	0.0737	0.0671
mapa5_11.txt	0.5651	0.2919	0.1671	0.1254	0.1048	0.0747	0.0663
mapa7_11.txt	3.7385	1.6625	0.7978	0.5330	0.4040	0.2115	0.1482
mapa15_5.txt	0.9867	0.6713	0.3151	0.2936	0.2293	0.1707	0.1921
mapa9_9.txt	21.2974	9.7176	4.5139	2.5687	1.8301	0.8365	0.6496
mapa5_15.txt	20.0154	10.1286	5.2823	3.4293	2.5740	1.3236	0.9502

Tabulka 3: Základní instance: OpenMP Taskový paralelismus [s]

Instance	2 vl.	4 vl.	8 vl.	12 vl.	16 vl.	32 vl.	48 vl.
mapa3_5.txt	0.0070	0.0075	0.0079	0.0088	0.0087	0.0107	0.0119
mapa5_5.txt	0.0130	0.0119	0.0122	0.0116	0.0119	0.0134	0.0150
mapa7_7.txt	0.2395	0.0793	0.0519	0.0512	0.0098	0.0125	0.0642
mapa3_11.txt	0.0504	0.0493	0.0321	0.0393	0.0383	0.0247	0.0265
mapa4_15.txt	0.5064	0.2366	0.0172	0.0314	0.0134	0.0130	0.0140
mapa7_10.txt	2.1421	0.0844	0.0399	0.0201	0.0172	0.0138	0.0159
mapa5_11.txt	0.5560	0.3165	0.1407	0.1033	0.0825	0.0516	0.0776
mapa7_11.txt	12.4940	1.5124	0.0595	0.0281	0.0386	0.0234	0.0249
mapa15_5.txt	3.3365	1.5015	0.8899	0.6866	0.4009	0.4612	0.3129
mapa9_9.txt	21.0660	7.1943	2.6988	1.7816	1.7330	1.2524	1.5645
mapa5_15.txt	22.4186	11.0079	4.7481	3.8678	2.8779	1.9953	1.2342

Tabulka 4: Základní instance: MPI implementace (při 48 vláknech na uzlu) [s]

Instance	1 uzel*	2 uzly	3 uzly	4 uzly
mapa3_5.txt	3.1561	3.9531	3.4037	3.2280
mapa5_5.txt	2.9004	4.6440	4.1610	3.4623
mapa7_7.txt	3.8634	5.2528	4.0338	3.6858
mapa3_11.txt	2.8926	4.9828	4.3498	3.7480
mapa4_15.txt	2.8856	5.9187	4.3626	4.5657
mapa7_10.txt	3.8631	6.8089	6.2802	5.7452
mapa5_11.txt	4.0210	7.7765	5.7464	4.9370
mapa7_11.txt	6.7410	31.9296	22.0765	21.1751
mapa15_5.txt	4.8595	10.0704	7.5578	6.3656
mapa9_9.txt	29.2734	23.1210	13.9836	24.9554
mapa5_15.txt	41.2743	85.9870	46.8671	35.8741

Analýza: U menších úloh se velmi zřetelně projevuje fenomén *paralelního zpomalení*. V tabulce s MPI výsledky si lze povšimnout, že jakmile program opustí lokální paměť 1 uzlu a zapojí síťovou komunikaci (2 a více uzlů), výpočetní čas se radikálně zhorší. Problémový prostor těchto instancí je natolik malý, že síťová latence a synchronizace ořezávací meze (Master-Slave overhead) trvá podstatně déle než samotný výpočet kombinací v procesoru. Srovnání přístupů uvnitř uzlu navíc ukazuje, že datový paralelismus (DATA) reaguje na menší instance stabilněji, zatímco taskový model (TASK) při vyšším počtu vláken (nad 16) naráží na režii spojenou se zakládáním a přepínáním kontextu vláken, čímž se vyčerpá množství užitečné práce dříve, než se dostaví zrychlení.

5.4 Náročné instance (vlastní generované mapy)

Skutečné přínosy paralelního škálování se naplno uplatňují až u složitých úloh. Následující dvě tabulky porovnávají výkony jednotlivých architektur na mapách se sekvenční dobou běhu v rozmezí 2 až 5 minut. Pro srovnání technologií DATA a TASK je zvolen vertikální dvousloupcový formát.

Tabulka 5: Náročné instance: Srovnání OpenMP (DATA vs. TASK)

Počet vláken	DATA [s]	TASK [s]
mapa7_12_hard.txt (Seq: 200.79 s)		
2	179.91	190.95
4	92.28	18.31
8	45.39	27.30
12	27.37	18.12
16	20.14	21.13
32	9.03	18.13
48	6.28	18.08
mapa9_10_hard.txt (Seq: 120.03 s)		
2	55.97	111.04
4	4.68	109.54
8	4.28	21.55
12	4.28	43.89
16	4.28	5.01
32	4.16	8.55
48	4.21	4.73
mapa10_10_hard.txt (Seq: 328.82 s)		
2	24.62	293.95
4	5.06	92.95
8	3.37	92.91
12	3.07	63.82
16	3.07	29.80
32	3.07	32.30
48	1.11	34.54

Analýza OpenMP: Tabulka 5 nabízí přímé srovnání efektivity uvnitř jednoho výpočetního uzlu. Z dat jasně vyplývá převaha datového paralelismu (DATA). Díky prealokaci a pevné struktuře úkolů ve frontě (**EnoughStates**) dokáže DATA stabilně škálovat i na plných 48 vláknech. Naproti tomu přístup s využitím direktivy **task** naráží na zásadní škálovací bariéru. Asynchronní otevírání velkého množství tasků přináší masivní paměťovou režii, kvůli které se na 48 vláknech výkon nezlepšuje, a mnohdy dokonce zpomaluje (viz čas 34.54 s pro **mapa10_10_hard**). Nečekané skoky ve zrychlení (např. z 55 s na 4.6 s u **mapa9_10_hard**) poukazují na anomálie prohledávání, kdy zapojení určitého počtu vláken náhodně umožnilo rychlejší objevení silné ořezávací meze.

Tabulka 6: Náročné instance: Časová složitost MPI (při 48 vláknech na uzlu) [s]

Instance	1 uzel (seq)	1 uzel (par)*	2 uzly	3 uzly	4 uzly
mapa7_12_hard.txt	200.79	7.04	49.33	26.42	16.78
mapa9_10_hard.txt	120.03	4.12	12.56	6.38	4.35
mapa10_10_hard.txt	328.82	1.19	137.65	64.04	43.03

**Poznámka: Běh na 1 uzlu z principu obchází distribuovanou paměť a síťovou MPI komunikaci. Program detekuje běh na jednom stroji a provede rovnou spuštění optimalizovaného datového paralelismu, aby předešel zbytečné režii.*

Analýza MPI: Tabulka 6 zachycuje distribuovaný běh (až 192 vláken). Přechod na 2 uzly přináší inicializační penalizaci za síťovou komunikaci, avšak další zapojování uzlů u velkých úloh čas prokazatelně zkracuje (z 137 s na 43 s pro `mapa10_10_hard`). Vhodně navržený MPI model tak dokáže síťovou latenci u objemných stavových prostorů úspěšně kompenzovat.

Při srovnání s OpenMP (Tabulka 5) však MPI verze se 4 uzly nedosahuje extrémně nízkých časů čistého datového paralelismu (DATA) na 1 fyzickém stroji (43.03 s vs. 1.11 s). Úzkým hrdlem je zpožděné šíření ořezávací meze. Zatímco ve sdílené paměti se nové minimum propíše téměř okamžitě (*atomic write*), v MPI putuje asynchronně přes Master uzel. Vzniklá prodleva vede k množství zbytečně vykonané práce ve slepých větvích. Přesto MPI implementace prokazuje spolehlivé škálování a na rozdíl od taskového modelu (TASK), který na těchto úlohách výkonnostně kolabuje, si drží vysoce stabilní a předvídatelný trend.

6 Analýza a hodnocení výkonnostních charakteristik

Na základě naměřených experimentů, zejména na složitých instancích, lze formulovat několik klíčových závěrů ohledně škálovatelnosti, efektivity a specifických vlastností paralelního algoritmu Branch & Bound.

6.1 Efektivita a škálovatelnost sdílené paměti (OpenMP)

Srovnání datového (DATA) a taskového (TASK) paralelismu jasně demonstruje vliv režie (*overhead*) na efektivitu výpočtu. Datový paralelismus vykazuje vynikající škálovatelnost. Díky předgenerování dostatečně velké fronty počátečních stavů (nastavení `EnoughStates`) a využití dynamického plánovače (`schedule(dynamic)`) dochází k rovnoměrnému vytížení vláken. Režie na správu vláken je minimální, protože vlákna provádějí převážně čistý sekvenční výpočet nad přiděleným podstromem. Naopak taskový paralelismus s rostoucím počtem vláken škálovat přestává. I přes ladění parametru `TASK_CUTOFF` dochází při zapojení 32 a 48 vláken k zahlcení fronty úkolů a masivní alokaci paměti pro nové stavy. U nejtěžší mapy (`mapa10_10_hard.txt`) se čas na 48 vláknech zastavil na 34.54 s, zatímco DATA verze dosáhla času 1.11 s.

6.2 Škálovatelnost distribuované paměti (MPI) a paralelní zpomalení

Hybridní MPI implementace (kombinující zasílání zpráv mezi uzly a OpenMP uvnitř uzlu) v praxi jasně demonstruje hardwarové limity distribuovaných výpočtů. U méně náročných instancí (např. `mapa7_12_hard.txt`) bylo při přechodu z jednoho na více uzlů pozorováno výrazné **paralelní zpomalení**. Důvodem je skutečnost, že síťová latence a režie spojená s inicializací zpráv zcela převýšila užitek ze samotné výpočetní práce.

Na velkých stavových prostorech (např. `mapa10_10_hard.txt`) naopak algoritmus napříč uzly dobře škáluje: čas klesal ze 137.65 s (2 uzly) až na 43.03 s (4 uzly). Současně se zde však naplno projevilo úzké hrdlo distribuovaných prohledávacích algoritmů: zpožděné šíření ořezávací meze. Zatímco ve sdílené paměti (OpenMP) se nové minimum propíše téměř okamžitě (*atomic write*), v MPI síti tato informace putuje asynchronně přes uzel. Vzniklá prodleva nutí vzdálené procesy vykonávat masivní množství zbytečné výpočetní práce ve slepých větvích, což vysvětluje, proč MPI klastr nedosahuje v absolutních číslech tak nízkých časů jako lokální datový paralelismus.

6.3 Superlineární zrychlení a anomálie prohledávání

V naměřených datech se na těžkých úlohách masivně projevuje **anomálie prohledávání** (Search Overhead Anomaly), typická pro paralelní B&B algoritmy. Nejvýraznějším příkladem je dosažené **superlineární zrychlení** u instance `mapa10_10_hard.txt` v OpenMP DATA verzi. Sekvenční algoritmus mapu řeší 328.82 s, avšak 48 vláken ji vyřeší za pouhých 1.11 s (zrychlení $S \approx 296$). Tento jev je způsoben odlišným pořadím prohledávání. Vygenerování úvodních stavů pomocí BFS a jejich zpracování asynchronním dynamickým plánovačem způsobilo, že některé z vláken velmi rychle narazilo na suboptimální či optimální řešení. Tím se drasticky snížila sdílená hodnota *best_cost*, což umožnilo ostatním vláknům okamžitě prořezat obrovské větve stromu, které by jinak sekvenční algoritmus (jdoucí striktně zleva doprava) musel zdlouhavě prohledávat.

7 Závěr

Cílem tohoto semestrálního projektu bylo navrhnout, implementovat a vyhodnotit paralelní algoritmy pro řešení NP-těžkého problému pokrývání mřížky quatrominy pomocí prohledávání stavového prostoru s ořezáváním (Branch & Bound). Úloha byla úspěšně realizována v sekvenční verzi a následně paralelizována pomocí OpenMP a rozhraní MPI pro běh na superpočítačovém klastru.

Z provedených experimentů a analýz vyplývá, že pro daný typ problému je zcela zásadní volba paralelizační strategie. Datový paralelismus prokázal výrazně vyšší efektivitu než taskový přístup, především díky eliminaci dynamické alokační režie a schopnosti bleskově sdílet ořezávací mez. Distribuované řešení v MPI sice potvrdilo schopnost škálovat a dekomponovat obrovské stavové stromy, nicméně narazilo na komunikační úzké hrdlo. Zpožděné šíření globálního minima po síti vede k nadbytečné výpočetní práci, kvůli které MPI v absolutním čase na zvolených instancích neporazilo čistý lokální paralelismus. Projekt rovněž na reálných datech jasně demonstroval existenci anomálií prohledávání, včetně masivního superlineárního zrychlení v důsledku dřívějšího objevení silné ořezávací meze.

8 Literatura

1. Dokumentace k rozhraní OpenMP: <https://www.openmp.org/>
2. Dokumentace k MPI knihovně: <https://www.open-mpi.org/doc/>
3. Studijní materiály a přednášky z předmětu NI-PDP, FIT ČVUT v Praze.